

6. Transfer of files from one Virtual Machine to another

Aim:

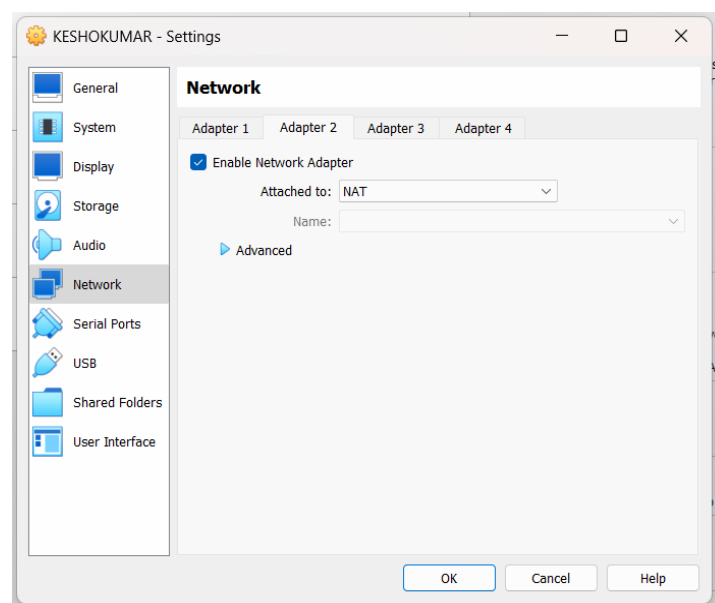
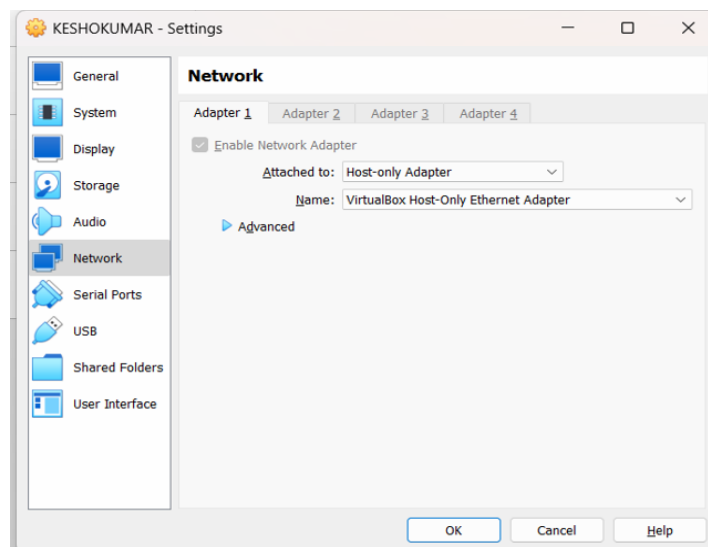
To transfer files from Ubuntu VM to Kali VM using NFS share, enabling read and write access on both ends.

Steps:

1. Configure VirtualBox Network

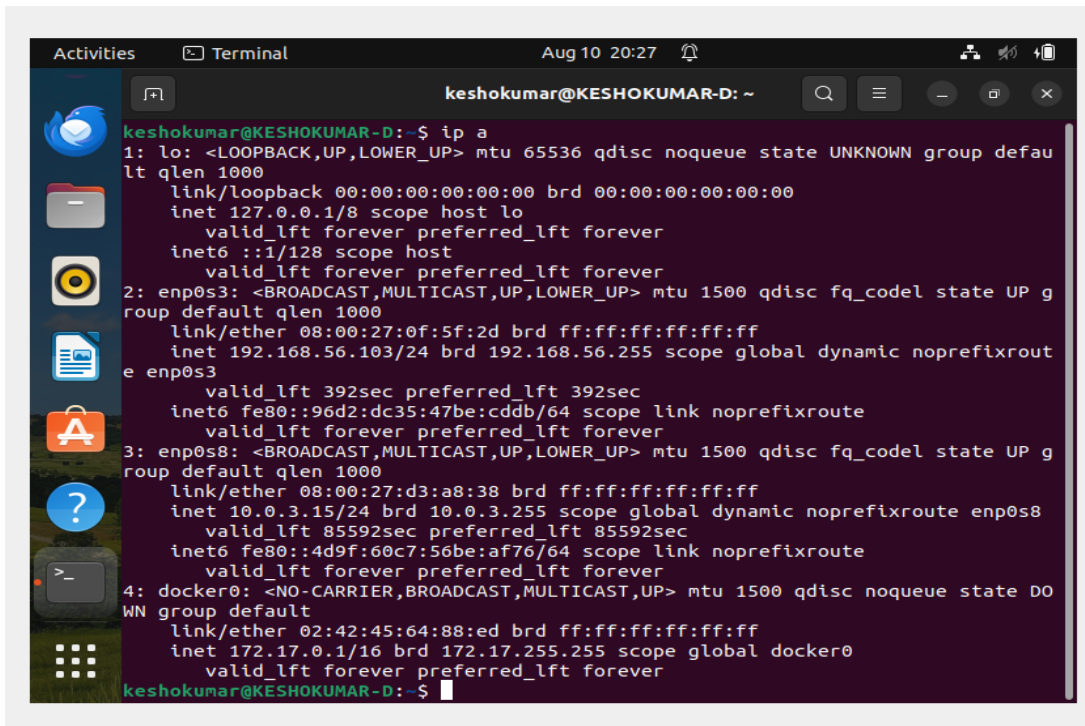
Set both VMs' network adapter to "**Host-only Adapter**" in **adapter 1** and **NAT in adapter 2** (for installing packages) so they are on the same subnet.

Go to settings -> network -> adapter 1-> Host only adapter and Adapter2-> NAT

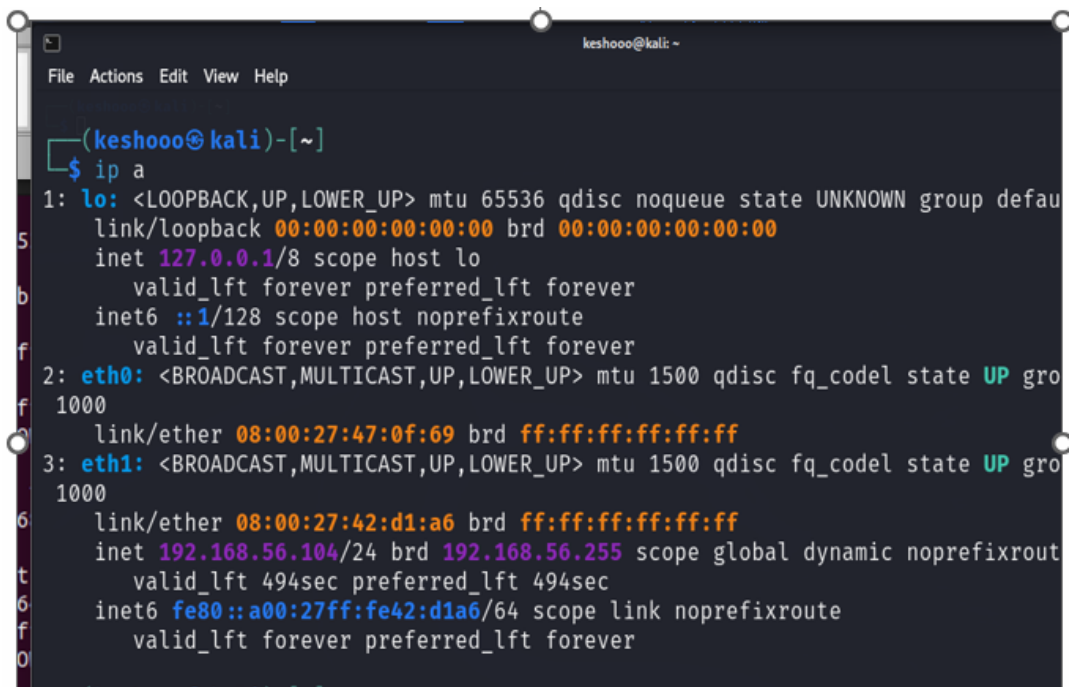


2. Check IP Addresses

On both ubuntu as well as kali **ip a** (check ip) Ensure both are in same subnet (eg 192.168.56.103/24 ubuntu and 192.168.56.104 kali) both in same subnet 192.168.56.0/24



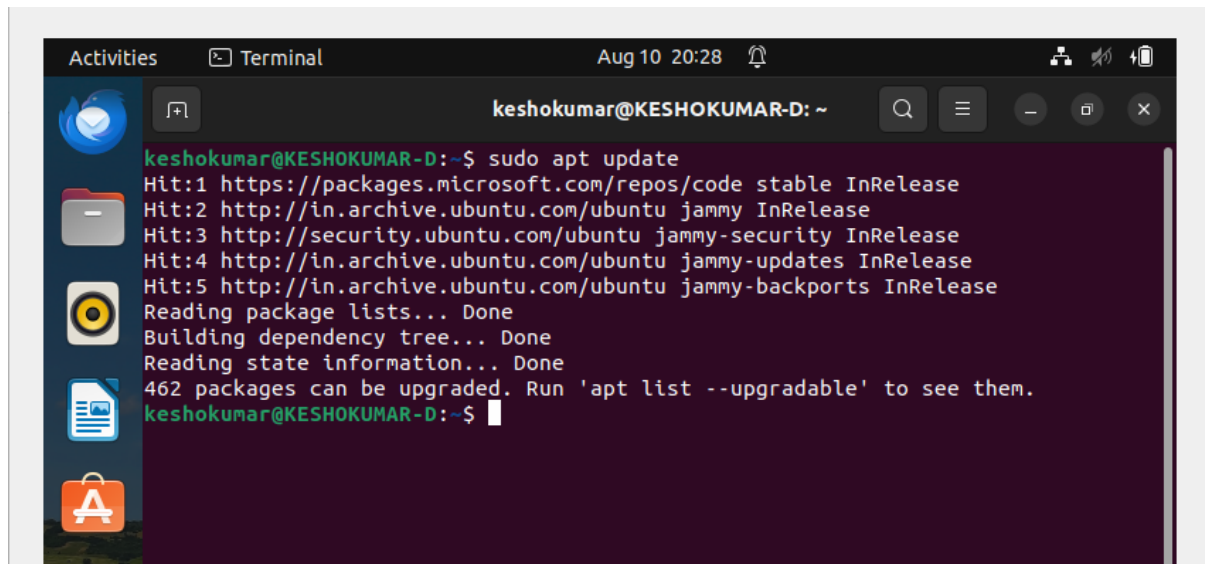
```
Activities Terminal Aug 10 20:27 keshokumar@KESHOKUMAR-D: ~
keshokumar@KESHOKUMAR-D:~$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defau
lt qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:0f:5f:2d brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixrou
te enp0s3
        valid_lft 392sec preferred_lft 392sec
    inet6 fe80::96d2:dc35:47be:cddb/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
3: enp0s8: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:d3:a8:38 brd ff:ff:ff:ff:ff:ff
    inet 10.0.3.15/24 brd 10.0.3.255 scope global dynamic noprefixroute enp0s8
        valid_lft 85592sec preferred_lft 85592sec
    inet6 fe80::4d9f:60c7:56be:af76/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
4: docker0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue state DO
WN group default
    link/ether 02:42:45:64:88:ed brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.1/16 brd 172.17.255.255 scope global docker0
        valid_lft forever preferred_lft forever
keshokumar@KESHOKUMAR-D:~$
```



```
keshooo@kali: ~
File Actions Edit View Help
(keshooo@kali)-[~]
$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group defau
lt qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
        valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:47:0f:69 brd ff:ff:ff:ff:ff:ff
3: eth1: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP gro
up default qlen 1000
    link/ether 08:00:27:42:d1:a6 brd ff:ff:ff:ff:ff:ff
    inet 192.168.56.104/24 brd 192.168.56.255 scope global dynamic noprefixrou
te eth1
        valid_lft 494sec preferred_lft 494sec
    inet6 fe80::a00:27ff:fe42:d1a6/64 scope link noprefixroute
        valid_lft forever preferred_lft forever
```

3. Set up NFS Server on Ubuntu VM(Here ubuntu act as server)

❖ Sudo apt update

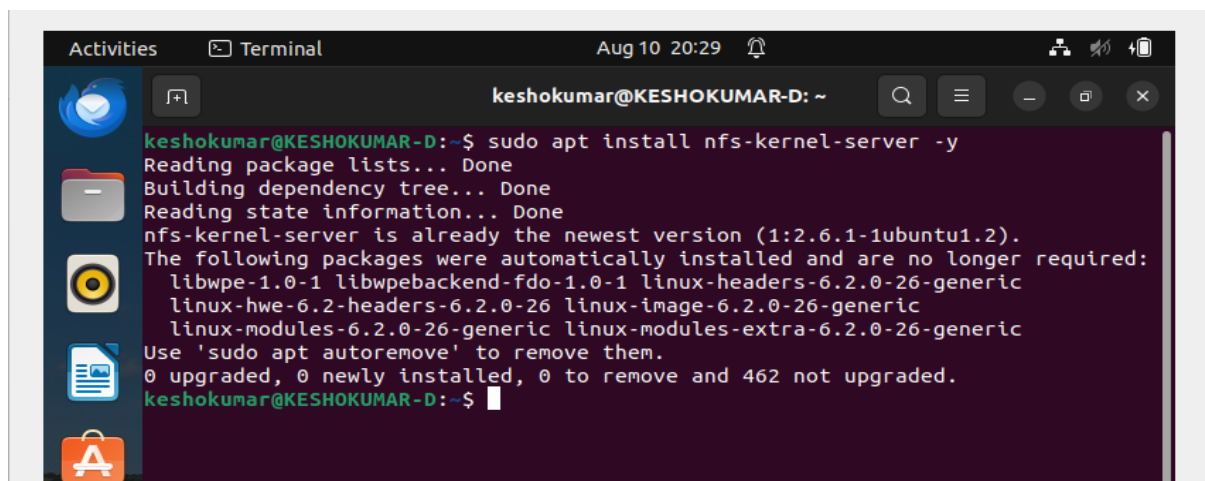


```
keshokumar@KESHOKUMAR-D: ~  
keshokumar@KESHOKUMAR-D:~$ sudo apt update  
Hit:1 https://packages.microsoft.com/repos/code stable InRelease  
Hit:2 http://in.archive.ubuntu.com/ubuntu jammy InRelease  
Hit:3 http://security.ubuntu.com/ubuntu jammy-security InRelease  
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-updates InRelease  
Hit:5 http://in.archive.ubuntu.com/ubuntu jammy-backports InRelease  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
462 packages can be upgraded. Run 'apt list --upgradable' to see them.  
keshokumar@KESHOKUMAR-D:~$
```

❖ Update and install NFS server:

sudo apt update

sudo apt install nfs-kernel-server -y



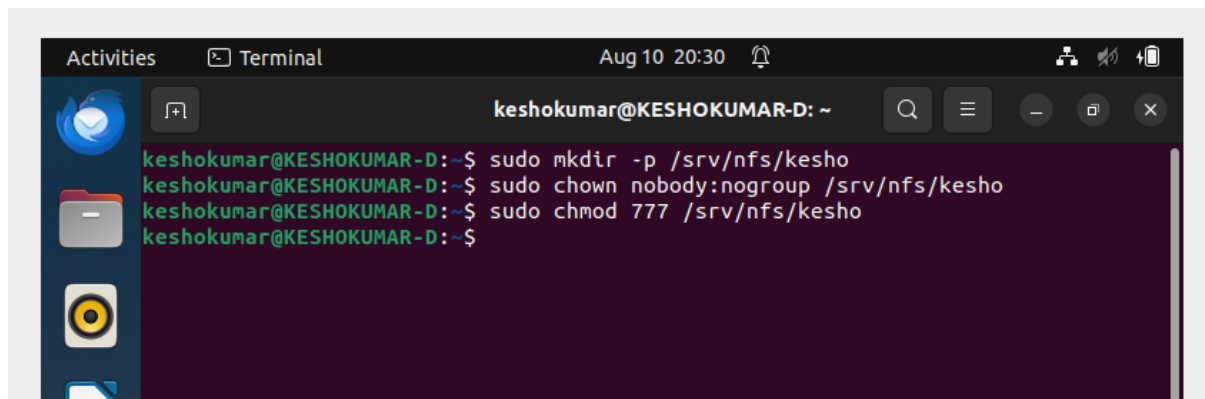
```
keshokumar@KESHOKUMAR-D: ~  
keshokumar@KESHOKUMAR-D:~$ sudo apt install nfs-kernel-server -y  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
nfs-kernel-server is already the newest version (1:2.6.1-1ubuntu1.2).  
The following packages were automatically installed and are no longer required:  
  libwpe-1.0-1 libwpebackend-fdo-1.0-1 linux-headers-6.2.0-26-generic  
  linux-hwe-6.2-headers-6.2.0-26 linux-image-6.2.0-26-generic  
  linux-modules-6.2.0-26-generic linux-modules-extra-6.2.0-26-generic  
Use 'sudo apt autoremove' to remove them.  
0 upgraded, 0 newly installed, 0 to remove and 462 not upgraded.  
keshokumar@KESHOKUMAR-D:~$
```

❖ Create folder to share:

`sudo mkdir -p /srv/nfs/sharedfolder` (or any folder name)

❖ Set permissions:

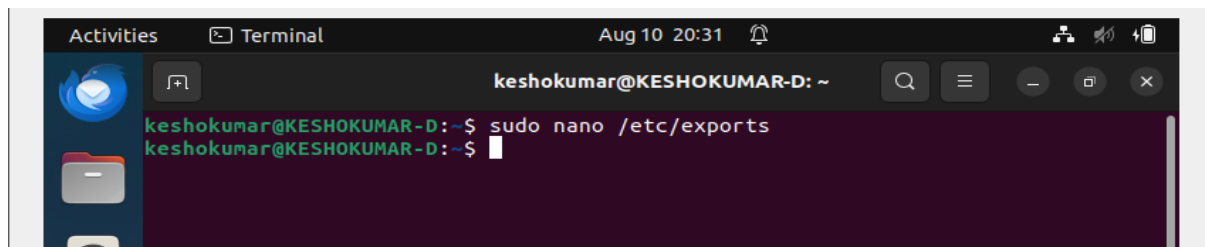
- `sudo chown nobody:nogroup /srv/nfs/sharedfolder`
- `sudo chmod 777 /srv/nfs/sharedfolder`



```
keshokumar@KESHOKUMAR-D: ~  
keshokumar@KESHOKUMAR-D:~$ sudo mkdir -p /srv/nfs/kesho  
keshokumar@KESHOKUMAR-D:~$ sudo chown nobody:nogroup /srv/nfs/kesho  
keshokumar@KESHOKUMAR-D:~$ sudo chmod 777 /srv/nfs/kesho  
keshokumar@KESHOKUMAR-D:~$
```

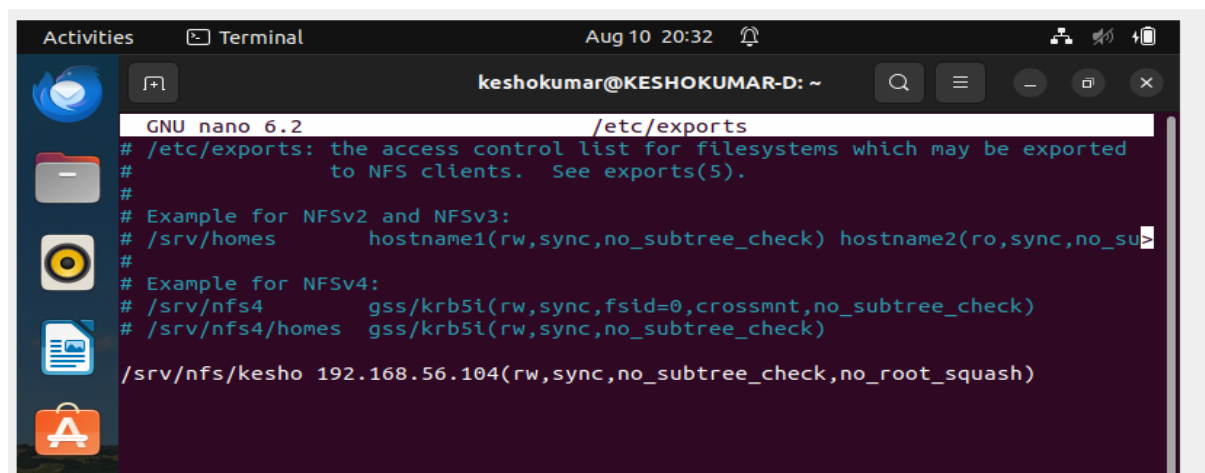
❖ Edit exports file to allow Kali IP access:

`sudo nano /etc/exports` (it open in nano editor)



```
keshokumar@KESHOKUMAR-D: ~  
keshokumar@KESHOKUMAR-D:~$ sudo nano /etc/exports  
keshokumar@KESHOKUMAR-D:~$
```

Add `/srv/nfs/sharedfolder < kali ip >(rw, sync, no_subtree_check, no_root_squash)`

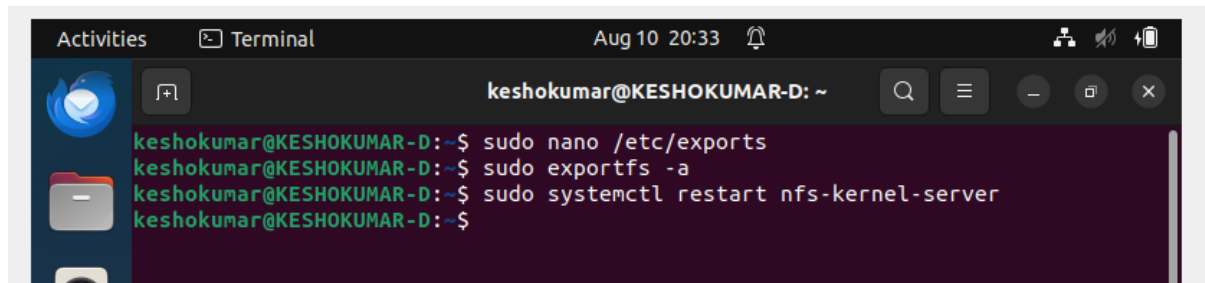


```
GNU nano 6.2 /etc/exports  
# /etc/exports: the access control list for filesystems which may be exported  
# to NFS clients. See exports(5).  
#  
# Example for NFSv2 and NFSv3:  
# /srv/homes hostname1(rw,sync,no_subtree_check) hostname2(ro,sync,no_su  
#  
# Example for NFSv4:  
# /srv/nfs4 gss/krb5i(rw,sync,fsid=0,crossmnt,no_subtree_check)  
# /srv/nfs4/homes gss/krb5i(rw,sync,no_subtree_check)  
/srv/nfs/kesho 192.168.56.104(rw,sync,no_subtree_check,no_root_squash)
```

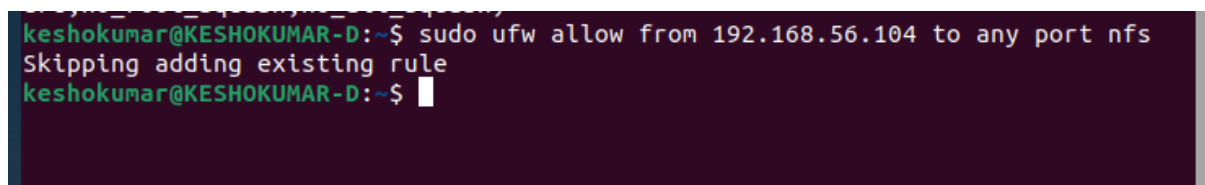
Click `ctrl+o` and enter and `ctrl x` (to save and came out of nano editor)

❖ Export and restart NFS server:

```
sudo exportfs -a  
sudo systemctl restart nfs-kernel-server
```



```
Activities Terminal Aug 10 20:33 keshokumar@KESHOKUMAR-D: ~  
keshokumar@KESHOKUMAR-D:~$ sudo nano /etc/exports  
keshokumar@KESHOKUMAR-D:~$ sudo exportfs -a  
keshokumar@KESHOKUMAR-D:~$ sudo systemctl restart nfs-kernel-server  
keshokumar@KESHOKUMAR-D:~$
```

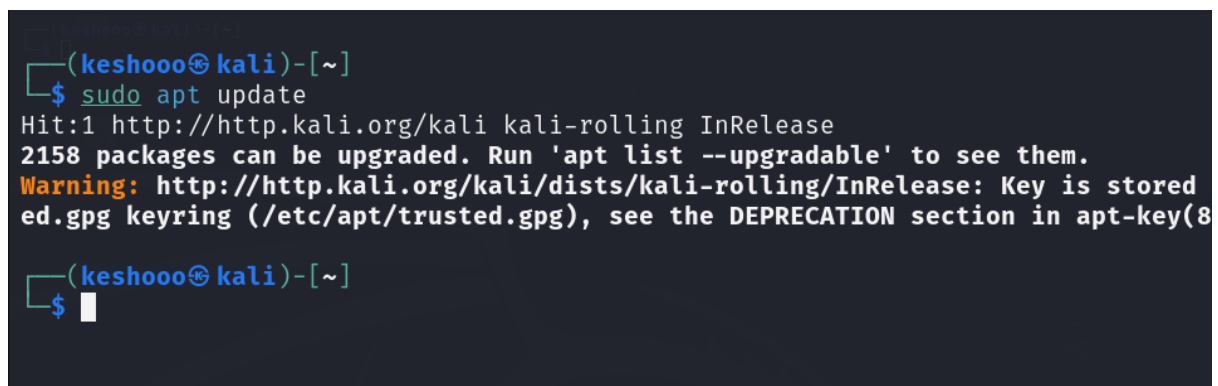
❖ Allow/disable the firewall for the kali linux (skipping shown already firewall in disable mode)

```
keshokumar@KESHOKUMAR-D:~$ sudo ufw allow from 192.168.56.104 to any port nfs  
Skipping adding existing rule  
keshokumar@KESHOKUMAR-D:~$
```

4. Set up NFS Client on Kali VM**❖ Install NFS client:**

- sudo apt update
- sudo apt install nfs-common -y

(here apt update not work because of network is in off so go to wired connection top right corner change the wired connection(NAT) and after above two command change again the wired connection1(Host only adapter)



```
(keshoooo@kali)-[~]  
$ sudo apt update  
Hit:1 http://http.kali.org/kali kali-rolling InRelease  
2158 packages can be upgraded. Run 'apt list --upgradable' to see them.  
Warning: http://http.kali.org/kali/dists/kali-rolling/InRelease: Key is stored  
ed.gpg keyring (/etc/apt/trusted.gpg), see the DEPRECATION section in apt-key(8)  
(keshoooo@kali)-[~]  
$
```

```
(keshooo@kali)-[~]
$ sudo apt install nfs-common -y
nfs-common is already the newest version (1:2.8.3-1).
Summary:
  Upgrading: 0, Installing: 0, Removing: 0, Not Upgrading: 2158

(keshooo@kali)-[~]
$
```

❖ Create mount point:

sudo mkdir -p /mnt/nfs/sharedfolder

```
(keshooo@kali)-[~]
$ sudo mkdir -p /mnt/nfs/kesho

(keshooo@kali)-[~]
$
```

❖ Mount the NFS share:

sudo mount ubuntu ip:/srv/nfs/dir_name /mnt/nfs/dir_name

eg:

sudo mount 192.168.56.103:/srv/nfs/sharedfolder /mnt/nfs/sharedfolder

```
(keshooo@kali)-[~]
$ sudo mount 192.168.56.103:/srv/nfs/kesho /mnt/nfs/kesho

(keshooo@kali)-[~]
$
```

❖ Verify:

df -h | grep nfs

ls -l /mnt/nfs/sharedfolder

```
(keshooo@kali)-[~]
$ df -h | grep nfs
192.168.56.103:/srv/nfs/kesho 34G 27G 5.3G 84% /mnt/nfs/kesho

(keshooo@kali)-[~]
$ ls -l /mnt/nfs/kesho
total 0

(keshooo@kali)-[~]
$
```

5. Test File Transfer

❖ On Ubuntu VM, create file:

echo "Hello from Ubuntu" | sudo tee /srv/nfs/dir_name/ubuntu_test.txt

```
keshokumar@KESHOKUMAR-D:~$ echo "Hello from Ubuntu" | sudo tee /srv/nfs/kesho/u
buntu_test.txt
Hello from Ubuntu
keshokumar@KESHOKUMAR-D:~$
```

❖ On Kali VM, verify(READ OPERATION)

ls -l /mnt/nfs/sharedfolder

cat /mnt/nfs/sharedfolder/ubuntu_test.txt

```
(keshooo@kali)-[~]
$ cat /mnt/nfs/kesho/ubuntu_test.txt
Hello from Ubuntu

(keshooo@kali)-[~]
$
```

❖ On Kali VM, (WRITE OPERATION):

```
(keshooo@kali)-[~]  
$ echo "Hello from Kali" | sudo tee /mnt/nfs/kesho/kali_test.txt  
Hello from Kali  
(keshooo@kali)-[~]  
$
```

❖ On Ubuntu VM, verify

```
ls -l /srv/nfs/sharedfolder
```

```
cat /srv/nfs/sharedfolder/kali_test.txt
```

```
keshokumar@KESHOKUMAR-D:~$ cat /srv/nfs/kesho/kali_test.txt  
Hello from Kali  
keshokumar@KESHOKUMAR-D:~$
```

METHOD 2

SHARED FOLDER

Step 1: Create a Shared Folder on Your Host

1. On your host machine (Windows/Linux):

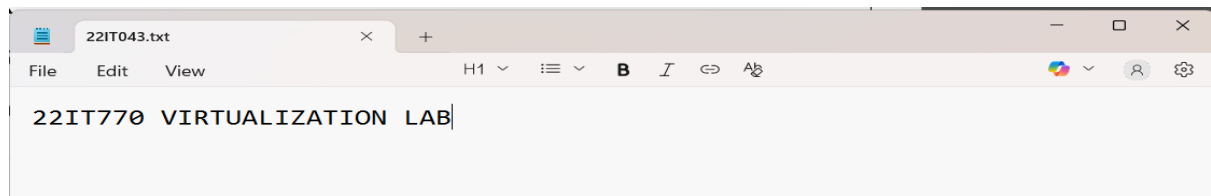
- Create a folder, e.g., C:\VMShared



- Add a test file (22IT043.txt) to confirm later.

DATE:

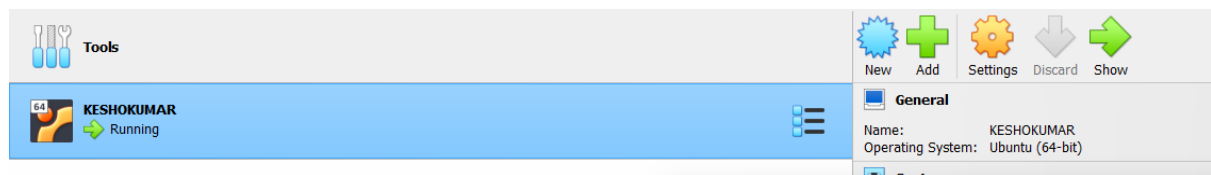
Reg.no_name



Step 2: Enable Shared Folder in VirtualBox (for Both VMs)

Repeat this for both Ubuntu VMs.

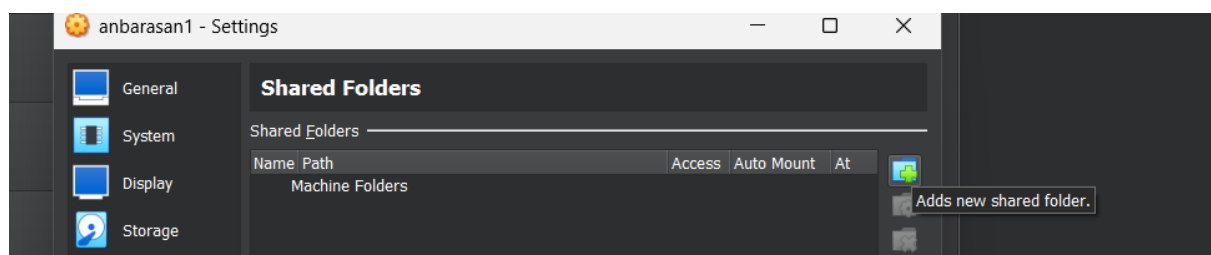
1. Shut down the VM if it's running.
2. In VirtualBox Manager, select your VM.



3. Click Settings Go to Shared Folders tab.



4. Click "+" (Add new shared folder).

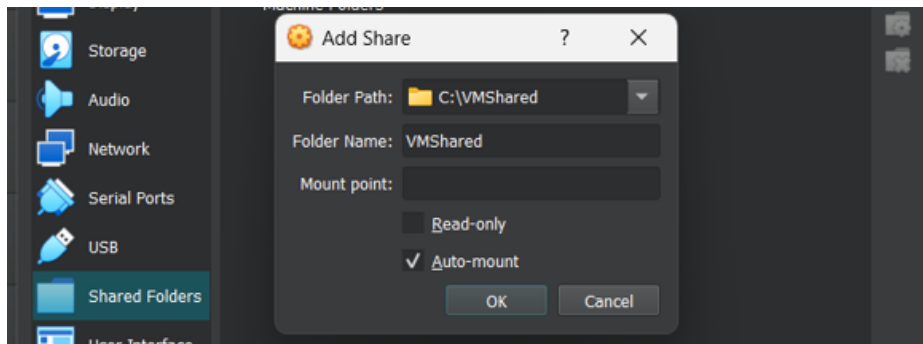


5. Set:

- Folder Path: Select C:\VMShared (or wherever you created it).
- Folder Name: VMShared (updates once path is entered)
- Check(Tick) Auto-mount

DATE:

Reg.no_name



Click OK.

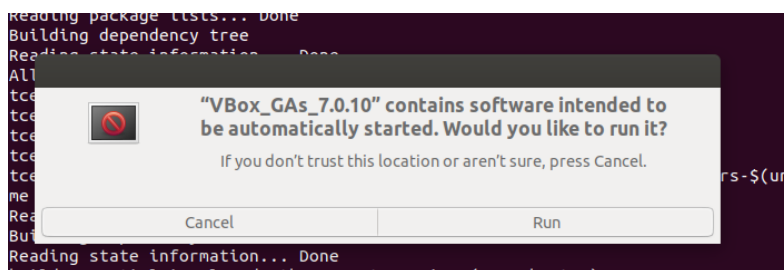
6. Then, again Click OK (at bottom) to save.

Do this for both VMs

VM2:

3.In VirtualBox menu (inside VM window), go to: *Devices* → *Insert Guest Additions CD image*

VM1:



DATE:

Reg.no_name

Now use the command: *ls /media*

- It lists all the directories (i.e., mounted devices or drives) under the /media directory.

```
tce@tce-VirtualBox:~$ ls /media
sf_VMShared  tce
tce@tce-VirtualBox:~$
```

Therefore, in VM1 *sf_VMShared* folder is there in */media* directory.

VM2: SAME PROCEDURE FOR virtualbox 2

Run the command: *ls /media*

```
ubuntu@ubuntu1804:~$ ls /media
sf_VMShared  ubuntu
ubuntu@ubuntu1804:~$
```

Therefore, in VM2 also *sf_VMShared* folder is there in */media* directory.

Step 4: Access the Shared Folder

After rebooting each VM, the shared folder should be mounted automatically at:
/media/sf_VMShared

To check, run the command : *ls /media*

VM1 :

```
tce@tce-VirtualBox:~$ ls /media/
sf_VMShared  tce
tce@tce-VirtualBox:~$
```

VM2 :

```
ubuntu@ubuntu1804:~$ ls /media
sf_VMShared  ubuntu
ubuntu@ubuntu1804:~$
```

Run the command : *ls /media/sf_VMShared*

VM1 :

```
tce@tce-VirtualBox:~$ ls /media/sf_VMShared
ls: cannot open directory '/media/sf_VMShared': Permission denied
tce@tce-VirtualBox:~$
```

DATE:

Reg.no_name

VM2 :

```
ubuntu@ubuntu1804:~$ ls /media/sf_VMShared  
ls: cannot open directory '/media/sf_VMShared': Permission denied
```

You should see the 22IT043.txt file from your host.

But in VM1 and VM2 , it asks for permission to list the files of sf_VMShared folder.

Step 5: Fix Permissions (if Permission Denied)

You need to add your user to the vboxsf group:

sudo usermod -aG vboxsf \$USER

```
tce@tce-VirtualBox:~$ sudo usermod -aG vboxsf $USER  
[sudo] password for tce:  
tce@tce-VirtualBox:~$
```

```
ubuntu@ubuntu1804:~$ sudo usermod -aG vboxsf $USER
```

Then reboot again:

sudo reboot

Now try again:

ls /media/sf_VMShared``

```
tce@tce-VirtualBox:~$ ls /media/sf_VMShared  
22IT043.txt
```

```
ubuntu@ubuntu1804:~$ ls /media/sf_VMShared  
22IT043.txt
```

You should now have full access.

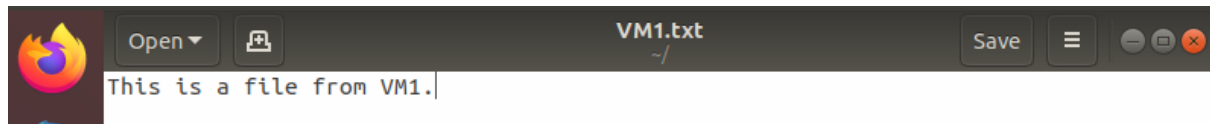
Step 6: Transfer Files Between VMs

- Trying to print the content of 22IT043.txt file from VM1

```
22IT770 VIRTUALIZATION TECHNOLOGIES LAB tce@tce-VirtualBox:~$
```

- Now in VM1, copy a file to /media/sf_VMShared folder,

```
tce@tce-VirtualBox:~$ touch VM1.txt
tce@tce-VirtualBox:~$ gedit VM1.txt
```



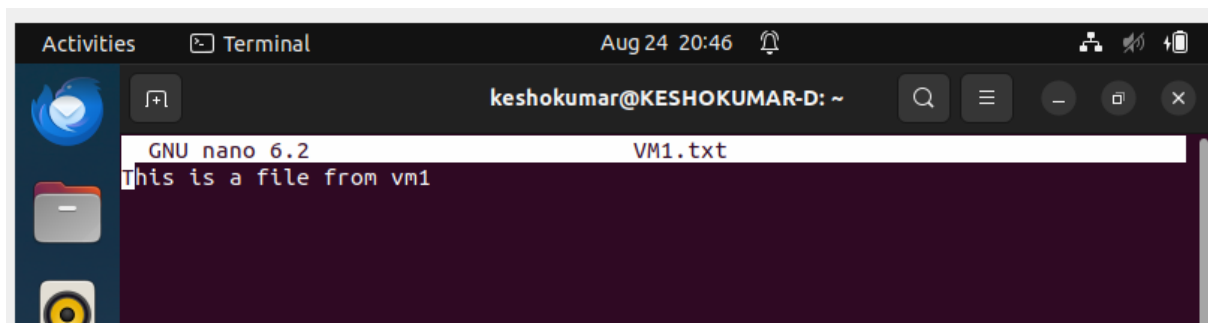
cp VM1.txt /media/sf_VMShared/

- Switch to VM2, and check the shared folder:

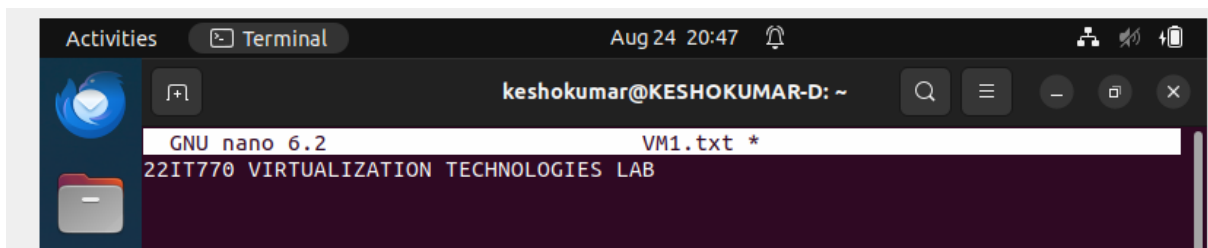
ls /media/sf_VMShared

The VM1.txt file will appear — you can copy it out to any other location and also you can edit files in shared folder location itself. Any changes done in shared folder using one VM will also be reflected to the other VM.

```
ubuntu@ubuntu1804:~$ cp /media/sf_VMShared/VM1.txt /home/ubuntu
ubuntu@ubuntu1804:~$ nano VM1.txt
```



Also 22IT043.txt file is accessible in VM2,



DATE:

Reg.no_name

METHOD 3(SSH Secure Shell)

Step 1: Install SSH

On both VMs (VM1 & VM2):

sudo apt update

sudo apt install openssh-server -y

```
tce@tce-VirtualBox:~$ sudo apt install openssh-server -y
[sudo] password for tce:
Reading package lists... Done
Building dependency tree
Reading state information... Done
openssh-server is already the newest version (1:7.6p1-4ubuntu0.7).
The following packages were automatically installed and are no longer required:
```

```
ubuntu@ubuntu1804:~$ sudo apt install openssh-server -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following additional packages will be installed:
  openssh-client openssh-sftp-server
```

Check SSH status:

sudo systemctl status ssh

```
tce@tce-VirtualBox:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enab
   Active: active (running) since Fri 2025-08-22 19:11:46 IST; 2h 12min ago
     Main PID: 779 (sshd)
```

```
ubuntu@ubuntu1804:~$ sudo systemctl status ssh
● ssh.service - OpenBSD Secure Shell server
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2025-08-22 12:10:18 EDT; 8min ago
     Main PID: 2715 (sshd)
```

Step 2: Get Server's IP

On VM1 (Server, username = tce):*ip a*

```
inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
   valid_lft 364sec preferred_lft 364sec
```

Allow firewall settings for SSH to avoid Connection timed out error.

sudo ufw allow 22/tcp

sudo ufw reload

DATE:

Reg.no_name

```
tce@tce-VirtualBox:~$ sudo ufw allow 22/tcp
Rule added
Rule added (v6)
tce@tce-VirtualBox:~$ sudo ufw reload
Firewall reloaded
tce@tce-VirtualBox:~$
```

Step 3: Connect from VM2 to VM1

On VM2 (Client, username = ubuntu):

`ssh tce@192.168.56.103`

```
ubuntu@ubuntu1804:~$ ssh tce@192.168.56.103
The authenticity of host '192.168.56.103 (192.168.56.103)' can't be established.
ECDSA key fingerprint is SHA256:ImaQYLvZCqfB/6sZmSQmIJta7VCyk+4M5Sfz0gEL2V4.
Are you sure you want to continue connecting (yes/no)? yes
Warning: Permanently added '192.168.56.103' (ECDSA) to the list of known hosts.
tce@192.168.56.103's password:
Welcome to Ubuntu 18.04.6 LTS (GNU/Linux 5.4.0-150-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

Expanded Security Maintenance for Infrastructure is not enabled.

1 update can be applied immediately.
To see these additional updates run: apt list --upgradable

289 additional security updates can be applied with ESM Infra.
Learn more about enabling ESM Infra service for Ubuntu 18.04 at
https://ubuntu.com/18-04

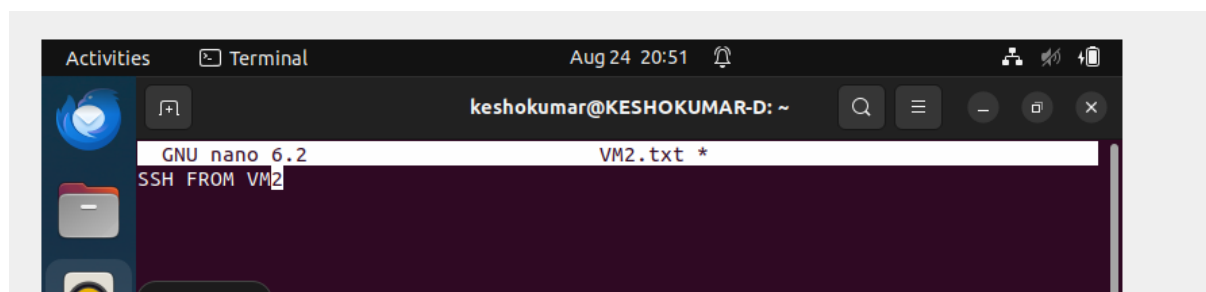
Your Hardware Enablement Stack (HWE) is supported until April 2023.
Last login: Sat Oct  5 19:15:29 2024
tce@tce-VirtualBox:~$
```

Enter tce's password now you are logged into VM1.

Step 4: Transfer Files with SCP (From VM2)

(a) Copy file from VM2 VM1

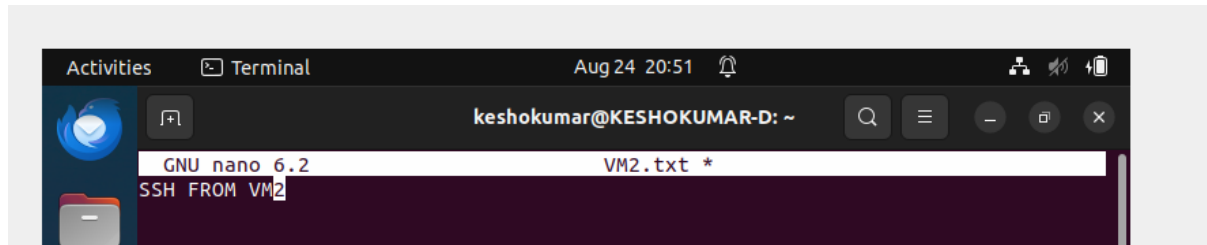
`scp file.txt tce@192.168.56.103:/home/tce/`



DATE:

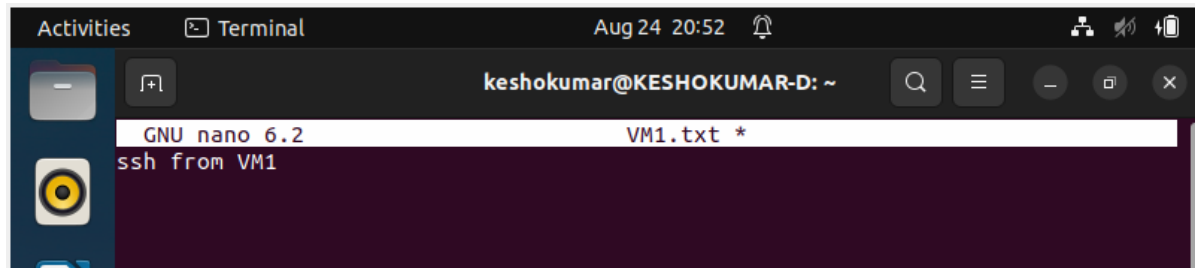
Reg.no_name

```
ubuntu@ubuntu1804:~$ scp VM2.txt tce@192.168.56.103:/home/tce/  
tce@192.168.56.103's password:  
VM2.txt 100% 13 4.3KB/s 00:00  
ubuntu@ubuntu1804:~$
```

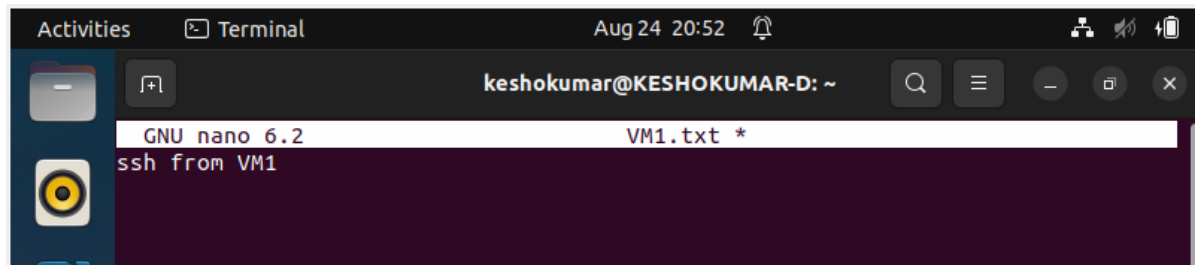


(b) Copy file from VM1 VM2

scp tce@192.168.56.103:/home/tce/serverfile.txt /home/ubuntu/



```
ubuntu@ubuntu1804:~$ scp tce@192.168.56.103:/home/tce/VM1.txt /home/ubuntu/  
tce@192.168.56.103's password:  
VM1.txt 100% 13 5.5KB/s 00:00
```



METHOD 4(FTP (File Transfer Protocol))

Step 1: Install FTP Server on VM1

(VM1 = server, VM2 = client)

On VM1:

sudo apt update

DATE:

Reg.no_name

sudo apt install vsftpd -y

```
tce@tce-VirtualBox:~$ sudo apt install vsftpd -y
[sudo] password for tce:
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages were automatically installed and are no longer required:
gir1.2-goa-1.0 gir1.2-snapd-1 libmemcached11 libmemcachedutil2 proftpd-doc
```

Enable and start the service:

sudo systemctl enable vsftpd

sudo systemctl start vsftpd

```
tce@tce-VirtualBox:~$ sudo systemctl enable vsftpd
Synchronizing state of vsftpd.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable vsftpd
tce@tce-VirtualBox:~$ sudo systemctl start vsftpd
tce@tce-VirtualBox:~$ sudo systemctl status vsftpd
● vsftpd.service - vsftpd FTP server
   Loaded: loaded (/lib/systemd/system/vsftpd.service; enabled; vendor preset: enabled)
   Active: active (running) since Fri 2025-08-22 23:00:53 IST; 9min ago
     Main PID: 5257 (vsftpd)
```

Step 2: Configure vsftpd

Edit the config file:

sudo nano /etc/vsftpd.conf

```
tce@tce-VirtualBox:~$ sudo nano /etc/vsftpd.conf
[sudo] password for tce:
```

Make sure these lines are set (uncomment or add if missing):

local_enable=YES

write_enable=YES

chroot_local_user=YES

allow_writeable_chroot=YES

Save (CTRL+S, CTRL+X)

DATE:

Reg.no_name

```
tce@tce-VirtualBox:~$ grep -v "^#" /etc/vsftpd.conf
listen=NO
listen_ipv6=YES
anonymous_enable=NO
local_enable=YES
write_enable=YES
dirmessage_enable=YES
use_localtime=YES
xferlog_enable=YES
connect_from_port_20=YES
chroot_local_user=YES
secure_chroot_dir=/var/run/vsftpd/empty
pam_service_name=vsftpd
rsa_cert_file=/etc/ssl/certs/ssl-cert-snakeoil.pem
rsa_private_key_file=/etc/ssl/private/ssl-cert-snakeoil.key
ssl_enable=NO

allow_writeable_chroot=YES
```

Restart the ftp server:

sudo systemctl restart vsftpd

```
tce@tce-VirtualBox:~$ sudo systemctl restart vsftpd
```

Step 3: Check VM1 IP Address

On VM1, run:

ip a

```
inet 192.168.56.103/24 brd 192.168.56.255 scope global dynamic noprefixroute enp0s3
    valid_lft 364sec preferred_lft 364sec
```

Allow firewall settings for FTP to avoid Connection timed out error.

sudo ufw allow 21/tcp

sudo ufw allow 20/tcp # (for data transfer)

sudo ufw status

```
tce@tce-VirtualBox:~$ sudo ufw allow 21/tcp
Rule added
Rule added (v6)
tce@tce-VirtualBox:~$ sudo ufw allow 20/tcp
Rule added
Rule added (v6)
tce@tce-VirtualBox:~$ sudo ufw reload
Firewall reloaded
```

Step 4: Connect from VM2 (FTP client)

On VM2, install ftp client:

sudo apt install ftp -y

```
ubuntu@ubuntu1804:~$ sudo apt install ftp -y
Reading package lists... Done
Building dependency tree
Reading state information... Done
```

Now connect:

ftp 192.168.56.103

Enter VM1's username (tce) and password when prompted.

```
ubuntu@ubuntu1804:~$ ftp 192.168.56.103
Connected to 192.168.56.103.
220 (vsFTPd 3.0.3)
Name (192.168.56.103:ubuntu): tce
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> █
```

Step 5: FTP Commands

Once connected, you can use:

- *ls* list files on VM1

```
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
drwxr-xr-x  3 1000    1000          4096 Oct 05  2024 Desktop
drwxr-xr-x  6 1000    1000          4096 Aug 23  00:21 Documents
drwxr-xr-x  3 1000    1000          4096 Aug 10  22:20 Downloads
drwxrwxr-x  2 1000    1000          4096 Aug 23  00:20 FTP1
drwxrwxr-x  2 1000    1000          4096 Aug 23  00:20 FTP2
drwxr-xr-x  2 1000    1000          4096 May 12  2022 Public
drwxr-xr-x  2 1000    1000          4096 May 12  2022 Templates
-rw-rw-r--  1 1000    1000           13 Aug 22  22:20 VM1.txt
-rw-rw-r--  1 1000    1000           13 Aug 22  22:17 VM2.txt
-rw-rw-r--  1 1000    1000          1301 Jul 19  22:19 anbarasan.class
-rw-rw-r--  1 1000    1000           625 Jul 19  22:18 anbarasan.java
drwxrwxr-x  2 1000    1000          4096 Aug 23  00:19 ubuntu images
226 Directory send OK.
```

- *cd folder* change directory on VM1

DATE:

Reg.no_name

```
ftp> cd FTP1
250 Directory successfully changed.
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-rw-r-- 1 1000 1000 17 Aug 23 00:18 ftp_vm1.txt
226 Directory send OK.
```

- *get filename* download file from VM1 VM2

```
ftp> get ftp_vm1.txt
local: ftp_vm1.txt remote: ftp_vm1.txt
200 PORT command successful. Consider using PASV.
150 Opening BINARY mode data connection for ftp_vm1.txt (17 bytes).
226 Transfer complete.
17 bytes received in 0.02 secs (0.7959 kB/s)
```

- *put filename* upload file from VM2 VM1

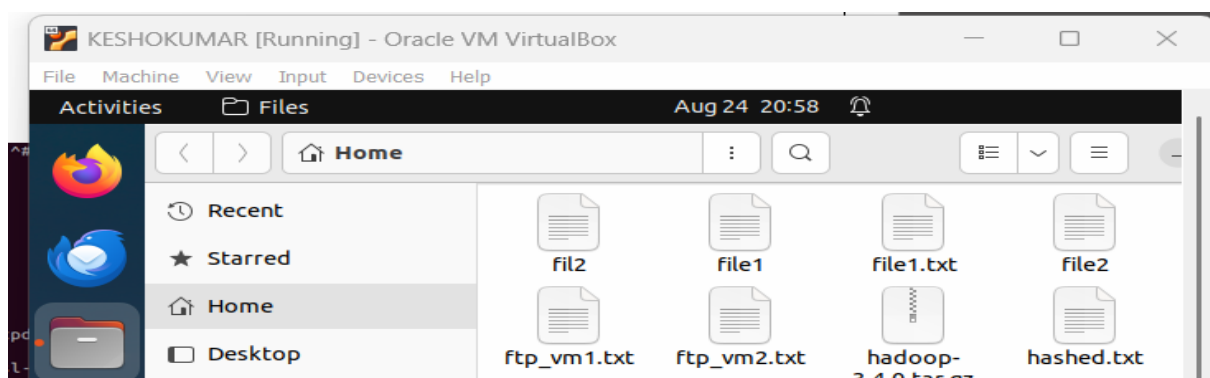
```
ftp> put ftp_vm2.txt
local: ftp_vm2.txt remote: ftp_vm2.txt
200 PORT command successful. Consider using PASV.
150 Ok to send data.
226 Transfer complete.
17 bytes sent in 0.00 secs (55.7099 kB/s)
ftp> ls
200 PORT command successful. Consider using PASV.
150 Here comes the directory listing.
-rw-rw-r-- 1 1000 1000 17 Aug 23 00:18 ftp_vm1.txt
-rw----- 1 1000 1000 17 Aug 23 00:26 ftp_vm2.txt
226 Directory send OK.
```

- *bye* exit

```
ftp> bye
221 Goodbye.
ubuntu@ubuntu1804:~$
```

Step 6: Verify

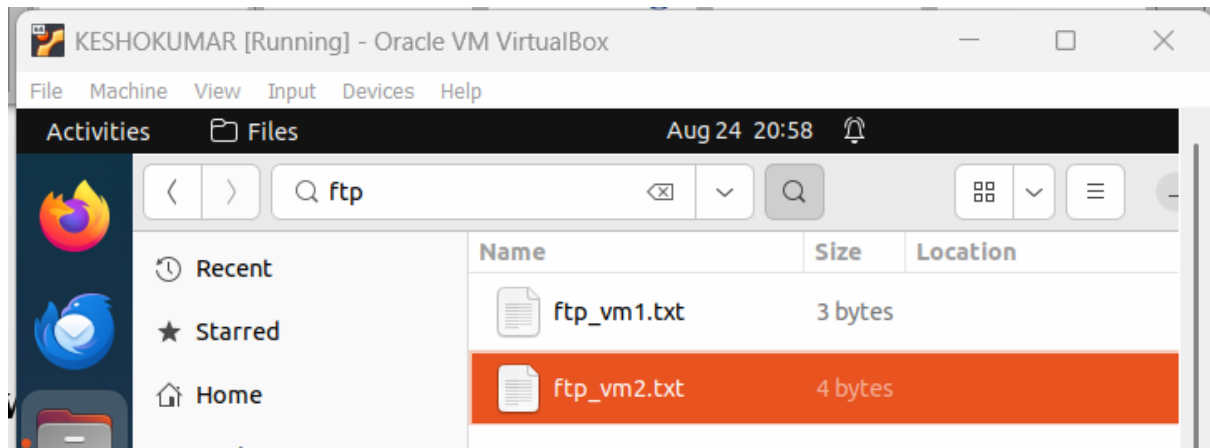
- ftp_vm1.txt should now be in VM2's current directory.



DATE:

Reg.no_name

- **ftp_vm2.txt** should now be uploaded to VM1's */home/FTP1/* directory.



RESULT:

Thus the file transfer between Ubuntu and Kali VMs is executed successfully and both VMs can read and write files in the shared folder seamlessly.